Lab 4: VibeCoding — Chat History

Name: Amogh Sharma | SRN: PES1UG24CS053
Repo: SETAPESU26/53_arm_wrestle (Arm Wrestle Showdown)
Tool: Claude (Claude desktop app) | Date: 30 Sep 2026

Prompt 1 — Amogh Sharma

I am working on a software engineering lab assignment involving a Python/Pygame arm wrestling game. I have attached the README, main.py, and game/game_engine.py.

According to the README, the player wins when arm_position reaches -100. However, when the player alternates the Left and Right arrow keys, the arm moves toward the computer instead. Please identify the root cause in handle_event(), explain it briefly, and provide the corrected method.

Do the changes in the code and let me know..

⚙ Ran command: cd "D:/SEM-5/SE/LAB/LAB-4" && ls -R | head -50 && cat game/game_engine.py
⚙ Ran command: cd "D:/SEM-5/SE/LAB/LAB-4/53_arm_wrestle" && cat -n game/game_engine.py && cat R
⚙ Edited game_engine.py
⚙ Ran command: cd "D:/SEM-5/SE/LAB/LAB-4/53_arm_wrestle" && SDL_VIDEODRIVER=dummy python -c " i

Claude

I fixed the bug in game_engine.py:36-47.

Root cause: The game treats `arm_position` as a number line. The computer wins at +100 and the player wins at −100. `update()` adds positive force every frame to push the arm toward the computer's side. But `handle_event()` also *added* `+4.2` on each valid Left/Right press. So every key press helped the computer instead of fighting it, and the player could never get to −100.

Fix: I changed `+=` to `-=` in both branches so each press moves the arm toward the player's winning side. The rest of the method is unchanged: the alternating-key check, the stamina cost and the stamina gate.

```
def handle_event(self, event):
    if self.game_state != "PLAYING":
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
        return

    if event.type == pygame.KEYDOWN:
        if self.stamina <= 10:
            return

        # Player wins at -target_limit, so player force must decrease arm_position
        # (the AI in update() adds positive force toward the computer's side).
        if event.key == pygame.K_LEFT:
            if self.last_key != pygame.K_LEFT:
                self.arm_position -= 4.2
                self.stamina = max(0.0, self.stamina - 2.0)
                self.last_key = pygame.K_LEFT
```

```
        elif event.key == pygame.K_RIGHT:
            if self.last_key != pygame.K_RIGHT:
                self.arm_position -= 4.2
                self.stamina = max(0.0, self.stamina - 2.0)
                self.last_key = pygame.K_RIGHT
```

Test: I ran the engine without opening a window and sent it alternating Left/Right presses, one game update per press. `arm_position` dropped to about −103, then the game ended with `winner = "PLAYER"` and `game_state = "GAME_OVER"`.

The README lists Task 3 and Task 4 with the same text (the exhaustion warning). You may want to ask your instructor what Task 4 was meant to be.

Prompt 2 — Amogh Sharma

The fix works correctly. Next, please implement Task 2 from the README.
Replace the constant AI force in `update()` with a dynamic surge mechanism with three states:

1. BUILDING – normal force while the AI's energy charges up
2. SURGE – roughly 2.5× force for 1–2 seconds
3. EXHAUSTED – roughly 0.3× force for 1–2 seconds, then back to BUILDING

The game runs at 60 FPS, so please use frame-based timers. Also display the AI's current state below the COMPUTER label, and make sure `reset()` restores all new variables. Please provide the complete updated `game_engine.py`.

⚙ Read game_engine.py

Claude

Now I'll make the edits: new state in `__init__`, the surge state machine in `update()`, reset of the new fields, and the state label in `render()`.

⚙ Edited game_engine.py
⚙ Edited game_engine.py
⚙ Edited game_engine.py
⚙ Edited game_engine.py

Claude

Now a headless test of the state cycle, the timing, reset, and render:

⚙ Ran command: cd "D:/SEM-5/SE/LAB/LAB-4/53_arm_wrestle" && SDL_VIDEODRIVER=dummy python -c " i

Claude

The state machine cycles correctly. Here is the full file:

⚙ Ran command: cd "D:/SEM-5/SE/LAB/LAB-4/53_arm_wrestle" && cat game/game_engine.py

Claude

I added the AI surge to game_engine.py. I ran the game without a window: the computer cycled through all three states, `reset()` put everything back to the start, and the screen drew with no errors. I didn't play it with a real window, so how hard it feels hasn't been checked by hand.

How it works:

State	Force	Length	What happens next
BUILDING	1.0× (normal)	Until energy reaches 100 (+0.55–0.85 energy per frame, about 2–2.5 s)	SURGE
SURGE	2.5×	Random 60–120 frames (1–2 s)	EXHAUSTED
EXHAUSTED	0.3×	Random 60–120 frames (1–2 s)	BUILDING

- **`update()`** calls a new `update_ai_state()` method every frame. The computer's push is now the old random force multiplied by the current state's number.
- **Display:** the state appears under the COMPUTER label: "BUILDING" in yellow, "SURGE!" in red, "EXHAUSTED" in grey.
- **`reset()`** now also restores `ai_energy`, `ai_state` and `ai_state_timer`.
- The three force numbers live together in `ai_force_multipliers`, so you can change the difficulty in one spot.

Test result: in 1,500 frames, the recorded state lengths were: - BUILDING: 143–144 frames - SURGE: 61–84 frames - EXHAUSTED: 65–99 frames

All are inside the ranges above. After `reset()`, the state went back to BUILDING and the energy, timer and arm position all went back to 0.

Complete updated `game_engine.py`:

```
import math
import random
import pygame


class GameEngine:

    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.arm_position = 0.0
        self.target_limit = 100.0
        self.last_key = None

        self.stamina = 100.0
        self.max_stamina = 100.0

        self.winner = None
        self.game_state = "PLAYING"
        self.ai_strength = 0.35

        # AI surge cycle: BUILDING -> SURGE -> EXHAUSTED -> BUILDING (timers in frames @ 60 FPS)
        self.ai_force_multipliers = {"BUILDING": 1.0, "SURGE": 2.5, "EXHAUSTED": 0.3}
        self.ai_max_energy = 100.0
        self.ai_energy = 0.0
        self.ai_state = "BUILDING"
        self.ai_state_timer = 0

        self.font_big = pygame.font.SysFont(None, 44)
```

```
        self.font_med = pygame.font.SysFont(None, 26)

    def handle_event(self, event):
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()
            return

        if event.type == pygame.KEYDOWN:
            if self.stamina <= 10:
                return

            # Player wins at -target_limit, so player force must decrease arm_position
            # (the AI in update() adds positive force toward the computer's side).
            if event.key == pygame.K_LEFT:
                if self.last_key != pygame.K_LEFT:
                    self.arm_position -= 4.2
                    self.stamina = max(0.0, self.stamina - 2.0)
                    self.last_key = pygame.K_LEFT
            elif event.key == pygame.K_RIGHT:
                if self.last_key != pygame.K_RIGHT:
                    self.arm_position -= 4.2
                    self.stamina = max(0.0, self.stamina - 2.0)
                    self.last_key = pygame.K_RIGHT

    def update(self):
        if self.game_state != "PLAYING":
            return

        self.update_ai_state()

        ai_variance = random.uniform(0.3, 1.0)
        multiplier = self.ai_force_multipliers[self.ai_state]
        self.arm_position += self.ai_strength * ai_variance * multiplier

        if self.stamina < self.max_stamina:
            self.stamina = min(self.max_stamina, self.stamina + 0.8)

        if self.arm_position <= -self.target_limit:
            self.winner = "PLAYER"
            self.game_state = "GAME_OVER"
        elif self.arm_position >= self.target_limit:
            self.winner = "COMPUTER"
            self.game_state = "GAME_OVER"

    def update_ai_state(self):
        if self.ai_state == "BUILDING":
            # Charges to full in roughly 2-3 seconds
            self.ai_energy = min(self.ai_max_energy, self.ai_energy + random.uniform(0.55, 0.85))
            if self.ai_energy >= self.ai_max_energy:
                self.ai_state = "SURGE"
                self.ai_state_timer = random.randint(60, 120)  # 1-2 seconds
        elif self.ai_state == "SURGE":
            self.ai_state_timer -= 1
            self.ai_energy = self.ai_max_energy * max(0, self.ai_state_timer) / 120
            if self.ai_state_timer <= 0:
```

```
                self.ai_state = "EXHAUSTED"
                self.ai_energy = 0.0
                self.ai_state_timer = random.randint(60, 120)  # 1-2 seconds
        elif self.ai_state == "EXHAUSTED":
            self.ai_state_timer -= 1
            if self.ai_state_timer <= 0:
                self.ai_state = "BUILDING"
                self.ai_state_timer = 0

    def reset(self):
        self.arm_position = 0.0
        self.stamina = 100.0
        self.last_key = None
        self.winner = None
        self.game_state = "PLAYING"
        self.ai_energy = 0.0
        self.ai_state = "BUILDING"
        self.ai_state_timer = 0

    def render(self, screen):
        screen.fill((25, 28, 35))

        title_surf = self.font_big.render("ARM WRESTLE SHOWDOWN", True, (240, 240, 240))
        screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 12))

        player_header = self.font_med.render("PLAYER", True, (80, 160, 255))
        computer_header = self.font_med.render("COMPUTER", True, (255, 100, 80))
        screen.blit(player_header, (60, 55))
        screen.blit(computer_header, (self.width - 150, 55))

        ai_state_colors = {
            "BUILDING": (230, 200, 90),
            "SURGE": (255, 60, 60),
            "EXHAUSTED": (140, 150, 170),
        }
        ai_label = "SURGE!" if self.ai_state == "SURGE" else self.ai_state
        ai_state_surf = self.font_med.render(ai_label, True, ai_state_colors[self.ai_state])
        screen.blit(ai_state_surf, (self.width - 150, 78))

        table_rect = pygame.Rect(40, 100, self.width - 80, 310)
        pygame.draw.rect(screen, (110, 50, 15), table_rect, border_radius=14)
        pygame.draw.rect(screen, (70, 30, 8), table_rect, width=5, border_radius=14)

        pygame.draw.line(screen, (45, 18, 4), (self.width // 2, 100), (self.width // 2, 410), 4)

        offset_x = (self.arm_position / self.target_limit) * 95
        hand_x = (self.width // 2) + int(offset_x)
        hand_y = 235

        p_shoulder = (70, 330)
        p_elbow = (140, 215)
        c_shoulder = (self.width - 70, 330)
        c_elbow = (self.width - 140, 215)

        pygame.draw.line(screen, (200, 145, 110), p_shoulder, p_elbow, 32)
        pygame.draw.line(screen, (215, 160, 125), p_elbow, (hand_x, hand_y), 26)
```

```
        pygame.draw.circle(screen, (185, 130, 95), p_elbow, 18)

        pygame.draw.line(screen, (170, 110, 85), c_shoulder, c_elbow, 32)
        pygame.draw.line(screen, (185, 125, 95), c_elbow, (hand_x, hand_y), 26)
        pygame.draw.circle(screen, (150, 95, 70), c_elbow, 18)

        pygame.draw.circle(screen, (225, 175, 140), (hand_x, hand_y), 24)
        pygame.draw.circle(screen, (160, 115, 85), (hand_x, hand_y), 24, width=3)

        stamina_label = self.font_med.render("STAMINA", True, (220, 220, 220))
        screen.blit(stamina_label, (40, 445))

        stamina_bg = pygame.Rect(140, 448, 240, 22)
        stamina_fill = pygame.Rect(140, 448, int(240 * (self.stamina / self.max_stamina)), 22)
        pygame.draw.rect(screen, (45, 50, 60), stamina_bg, border_radius=6)
        bar_color = (60, 210, 100) if self.stamina > 25 else (220, 60, 60)
        pygame.draw.rect(screen, bar_color, stamina_fill, border_radius=6)

        if self.game_state == "GAME_OVER":
            overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
            overlay.fill((0, 0, 0, 200))
            screen.blit(overlay, (0, 0))

            win_text = "PLAYER WINS THE MATCH!" if self.winner == "PLAYER" else "COMPUTER WINS!"
            color = (80, 240, 100) if self.winner == "PLAYER" else (240, 80, 80)
            text_surf = self.font_big.render(win_text, True, color)
            screen.blit(
                text_surf,
                (self.width // 2 - text_surf.get_width() // 2, self.height // 2 - 45)
            )

            restart_surf = self.font_med.render(
                "Press [R] to Rematch", True, (240, 240, 240)
            )
            screen.blit(
                restart_surf,
                (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 10)
            )
```

Run it with `python main.py` from the `53_arm_wrestle` folder to try it out.

Prompt 3 — Amogh Sharma

The surge mechanism works well. Now please implement Task 3 from the README (Task 4 is a duplicate of it).
When player stamina falls below the usable threshold of 10, input is disabled, but there is no visual feedback. Please add:

- a stamina bar that flashes red
- a blinking "EXHAUSTED!" label next to the bar
- a slight trembling effect on the player's arm

Please provide the complete updated `game_engine.py`.

⚙ Edited game_engine.py
⚙ Edited game_engine.py
⚙ Edited game_engine.py
⚙ Edited game_engine.py
⚙ Edited game_engine.py

Claude

Now testing: input gating, the flashing and label rendering, and whether stamina can actually drop below 10 during normal play.

⚙ Ran command: cd "D:/SEM-5/SE/LAB/LAB-4/53_arm_wrestle" && SDL_VIDEODRIVER=dummy python -c " i

Claude

All three effects render and input is blocked. There's one problem with how the game is balanced, which I explain below. Full file:

⚙ Ran command: cd "D:/SEM-5/SE/LAB/LAB-4/53_arm_wrestle" && cat game/game_engine.py

Claude

I added the exhaustion warning to game_engine.py, but with the current numbers you'll almost never see it. Stamina refills too fast for a person to drain it below 10 (details below). The warning itself works: when I set stamina to 5 by hand, key presses were ignored and all three effects showed up.

What was added: - **Threshold:** I added a new `is_exhausted()` method (`stamina <= exhaust_threshold`, where the threshold is 10). The key-press code and the drawing code both use it, so the warning appears exactly when input stops working. - **Flashing bar:** while exhausted, the whole stamina bar switches between bright and dark red every 150 ms. I flash the whole bar, not just the green fill, because below 10 stamina the fill is only a thin sliver. - **Blinking label:** a red "EXHAUSTED!" appears to the right of the bar, blinking every 300 ms. - **Trembling arm:** the player's elbow shakes by up to 3 pixels each frame and the shoulder by up to 1. The hand stays still because the computer's arm is attached to it too; shaking it would shake both arms. - **When it shows:** only during play, not behind the Game Over screen.

Test result: with stamina at 5, a Left press didn't move the arm. Over several frames the bar switched between red and grey, and the label appeared and disappeared. At 50 stamina the bar looked normal.

The balance problem: each press costs 2 stamina, but stamina refills by 0.8 per frame. At 60 FPS that's 48 per second, so you'd have to press more than 24 keys per second to fall behind. In my test: - At 10 or 20 presses per second, the lowest stamina reached was 98.8. - Only at 30 presses per second did it drop to 9.2.

A person can't press that fast, so your "after" video probably won't show the warning. The simplest fix is to lower the refill rate in `update()`. For example, 0.25 per frame (15 per second) lets steady fast pressing tire the player out. I didn't change it because that's a game design choice; tell me if you want it. To just record the effect, you could also temporarily set `self.stamina = 15.0` in `__init__`.

Complete updated `game_engine.py`:

```
import math
import random
import pygame


class GameEngine:
```

```
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.arm_position = 0.0
        self.target_limit = 100.0
        self.last_key = None

        self.stamina = 100.0
        self.max_stamina = 100.0
        self.exhaust_threshold = 10.0  # at or below this, player input is ignored

        self.winner = None
        self.game_state = "PLAYING"
        self.ai_strength = 0.35

        # AI surge cycle: BUILDING -> SURGE -> EXHAUSTED -> BUILDING (timers in frames @ 60 FPS)
        self.ai_force_multipliers = {"BUILDING": 1.0, "SURGE": 2.5, "EXHAUSTED": 0.3}
        self.ai_max_energy = 100.0
        self.ai_energy = 0.0
        self.ai_state = "BUILDING"
        self.ai_state_timer = 0

        self.font_big = pygame.font.SysFont(None, 44)
        self.font_med = pygame.font.SysFont(None, 26)

    def handle_event(self, event):
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()
            return

        if event.type == pygame.KEYDOWN:
            if self.is_exhausted():
                return

            # Player wins at -target_limit, so player force must decrease arm_position
            # (the AI in update() adds positive force toward the computer's side).
            if event.key == pygame.K_LEFT:
                if self.last_key != pygame.K_LEFT:
                    self.arm_position -= 4.2
                    self.stamina = max(0.0, self.stamina - 2.0)
                    self.last_key = pygame.K_LEFT
            elif event.key == pygame.K_RIGHT:
                if self.last_key != pygame.K_RIGHT:
                    self.arm_position -= 4.2
                    self.stamina = max(0.0, self.stamina - 2.0)
                    self.last_key = pygame.K_RIGHT

    def is_exhausted(self):
        return self.stamina <= self.exhaust_threshold

    def update(self):
        if self.game_state != "PLAYING":
            return
```

```
        self.update_ai_state()

        ai_variance = random.uniform(0.3, 1.0)
        multiplier = self.ai_force_multipliers[self.ai_state]
        self.arm_position += self.ai_strength * ai_variance * multiplier

        if self.stamina < self.max_stamina:
            self.stamina = min(self.max_stamina, self.stamina + 0.8)

        if self.arm_position <= -self.target_limit:
            self.winner = "PLAYER"
            self.game_state = "GAME_OVER"
        elif self.arm_position >= self.target_limit:
            self.winner = "COMPUTER"
            self.game_state = "GAME_OVER"

    def update_ai_state(self):
        if self.ai_state == "BUILDING":
            # Charges to full in roughly 2-3 seconds
            self.ai_energy = min(self.ai_max_energy, self.ai_energy + random.uniform(0.55, 0.85))
            if self.ai_energy >= self.ai_max_energy:
                self.ai_state = "SURGE"
                self.ai_state_timer = random.randint(60, 120)  # 1-2 seconds
        elif self.ai_state == "SURGE":
            self.ai_state_timer -= 1
            self.ai_energy = self.ai_max_energy * max(0, self.ai_state_timer) / 120
            if self.ai_state_timer <= 0:
                self.ai_state = "EXHAUSTED"
                self.ai_energy = 0.0
                self.ai_state_timer = random.randint(60, 120)  # 1-2 seconds
        elif self.ai_state == "EXHAUSTED":
            self.ai_state_timer -= 1
            if self.ai_state_timer <= 0:
                self.ai_state = "BUILDING"
                self.ai_state_timer = 0

    def reset(self):
        self.arm_position = 0.0
        self.stamina = 100.0
        self.last_key = None
        self.winner = None
        self.game_state = "PLAYING"
        self.ai_energy = 0.0
        self.ai_state = "BUILDING"
        self.ai_state_timer = 0

    def render(self, screen):
        screen.fill((25, 28, 35))

        title_surf = self.font_big.render("ARM WRESTLE SHOWDOWN", True, (240, 240, 240))
        screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 12))

        player_header = self.font_med.render("PLAYER", True, (80, 160, 255))
        computer_header = self.font_med.render("COMPUTER", True, (255, 100, 80))
        screen.blit(player_header, (60, 55))
        screen.blit(computer_header, (self.width - 150, 55))
```

```
ai_state_colors = {
    "BUILDING": (230, 200, 90),
    "SURGE": (255, 60, 60),
    "EXHAUSTED": (140, 150, 170),
}
ai_label = "SURGE!" if self.ai_state == "SURGE" else self.ai_state
ai_state_surf = self.font_med.render(ai_label, True, ai_state_colors[self.ai_state])
screen.blit(ai_state_surf, (self.width - 150, 78))

table_rect = pygame.Rect(40, 100, self.width - 80, 310)
pygame.draw.rect(screen, (110, 50, 15), table_rect, border_radius=14)
pygame.draw.rect(screen, (70, 30, 8), table_rect, width=5, border_radius=14)

pygame.draw.line(screen, (45, 18, 4), (self.width // 2, 100), (self.width // 2, 410), 4)

offset_x = (self.arm_position / self.target_limit) * 95
hand_x = (self.width // 2) + int(offset_x)
hand_y = 235

p_shoulder = (70, 330)
p_elbow = (140, 215)
c_shoulder = (self.width - 70, 330)
c_elbow = (self.width - 140, 215)

exhausted = self.is_exhausted() and self.game_state == "PLAYING"
ticks = pygame.time.get_ticks()

# Tremble the player's arm while exhausted. The hand stays put because the
# computer's arm is attached to it too.
if exhausted:
    p_shoulder = (p_shoulder[0] + random.randint(-1, 1), p_shoulder[1] + random.randint(-1, 1))
    p_elbow = (p_elbow[0] + random.randint(-3, 3), p_elbow[1] + random.randint(-3, 3))

pygame.draw.line(screen, (200, 145, 110), p_shoulder, p_elbow, 32)
pygame.draw.line(screen, (215, 160, 125), p_elbow, (hand_x, hand_y), 26)
pygame.draw.circle(screen, (185, 130, 95), p_elbow, 18)

pygame.draw.line(screen, (170, 110, 85), c_shoulder, c_elbow, 32)
pygame.draw.line(screen, (185, 125, 95), c_elbow, (hand_x, hand_y), 26)
pygame.draw.circle(screen, (150, 95, 70), c_elbow, 18)

pygame.draw.circle(screen, (225, 175, 140), (hand_x, hand_y), 24)
pygame.draw.circle(screen, (160, 115, 85), (hand_x, hand_y), 24, width=3)

stamina_label = self.font_med.render("STAMINA", True, (220, 220, 220))
screen.blit(stamina_label, (40, 445))

stamina_bg = pygame.Rect(140, 448, 240, 22)
stamina_fill = pygame.Rect(140, 448, int(240 * (self.stamina / self.max_stamina)), 22)
if exhausted:
    # Flash the whole bar, since the fill is only a sliver below 10 stamina
    flash_on = (ticks // 150) % 2 == 0
    bg_color = (150, 25, 25) if flash_on else (45, 50, 60)
    bar_color = (255, 70, 70) if flash_on else (120, 20, 20)
else:
```

```
            bg_color = (45, 50, 60)
            bar_color = (60, 210, 100) if self.stamina > 25 else (220, 60, 60)
        pygame.draw.rect(screen, bg_color, stamina_bg, border_radius=6)
        pygame.draw.rect(screen, bar_color, stamina_fill, border_radius=6)

        if exhausted and (ticks // 300) % 2 == 0:
            exhausted_surf = self.font_med.render("EXHAUSTED!", True, (255, 70, 70))
            screen.blit(exhausted_surf, (stamina_bg.right + 15, 450))

        if self.game_state == "GAME_OVER":
            overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
            overlay.fill((0, 0, 0, 200))
            screen.blit(overlay, (0, 0))

            win_text = "PLAYER WINS THE MATCH!" if self.winner == "PLAYER" else "COMPUTER WINS!"
            color = (80, 240, 100) if self.winner == "PLAYER" else (240, 80, 80)
            text_surf = self.font_big.render(win_text, True, color)
            screen.blit(
                text_surf,
                (self.width // 2 - text_surf.get_width() // 2, self.height // 2 - 45)
            )

            restart_surf = self.font_med.render(
                "Press [R] to Rematch", True, (240, 240, 240)
            )
            screen.blit(
                restart_surf,
                (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 10)
            )
```

Prompt 4 — Amogh Sharma

All features are working. Please do a brief final review of game_engine.py for edge cases, such as whether the stamina threshold check is consistent between input handling and the warning display, and whether everything resets correctly on rematch. Suggest only minimal fixes, if any.

⚙ Ran command: cd "D:/SEM-5/SE/LAB/LAB-4/53_arm_wrestle" && SDL_VIDEODRIVER=dummy python -c " i

Claude

I found one real problem. The threshold check and the rematch reset are both fine.

What's fine: - **The threshold is the same everywhere.** Key presses and the warning both use `is_exhausted()`. The warning also requires `game_state == "PLAYING"`, and that matches too, because presses are ignored outside PLAYING anyway (except R). - **Rematch resets everything that changes during play:** `arm_position`, `stamina`, `last_key`, `winner`, `game_state`, `ai_energy`, `ai_state` and `ai_state_timer`. The other variables are fixed settings that never change. - **Win detection is safe.** A key press can push the arm slightly past −100 (about −103 in testing), but the win is caught in the same frame's `update()`, and the hand is still drawn in a sensible place. The player and computer can't both win at once, because the checks are `if/elif`.

The problem: exhaustion only lasts a frame or two. Stamina refills by 0.8 each frame, so once it drops to 10 or below, it's back above 10 almost immediately. I checked this: - Starting from 5 stamina, the player was exhausted for only 7 frames (about 0.1 s). - A press that dropped stamina from 10.5 to 8.5 locked input for about 2 frames.

That means both the input lock and the warning disappear before anyone can see them. The flash cycle is 150 ms and the blink cycle is 300 ms, so the effects usually don't even get through one cycle. The README says input stays disabled "until stamina regenerates", which suggests a real recovery period.

Minimal fix: once the player is exhausted, keep them locked until stamina recovers to about 30. This needs a flag that is set when stamina drops too low and cleared when it recovers:

```
# __init__
self.exhaust_threshold = 10.0   # at or below this, player input is ignored...
self.recover_threshold = 30.0   # ...until stamina climbs back to this
self.player_exhausted = False

# replace is_exhausted()
def is_exhausted(self):
    return self.player_exhausted

# handle_event(), right after the stamina cost in each branch (or once after the if/elif)
if self.stamina <= self.exhaust_threshold:
    self.player_exhausted = True

# update(), after stamina regeneration
if self.player_exhausted and self.stamina >= self.recover_threshold:
    self.player_exhausted = False

# reset()
self.player_exhausted = False
```

With 0.8 refill per frame, the lock would last about 0.4 s. That's enough to see the flash and blink a couple of times. It still depends on the refill-rate issue from my last message: at the current rate, a player can't drain stamina to 10 in the first place. Lowering the refill to about 0.25 per frame fixes both at once, and the lock would then last about 1.3 s.

Small cleanups (optional): - `reset()` sets `self.stamina = 100.0` instead of `self.max_stamina`, so the two could drift apart if someone changes the maximum later. - `import math` is never used.

I haven't changed the file. Want me to apply the lock fix, and possibly the lower refill rate?

Prompt 5 — Amogh Sharma

Yes, please apply both the exhaustion lock fix and the lower stamina refill rate (0.25 per frame). Also apply the two small cleanups: use self.max_stamina in reset() and remove the unused import math. Please share the final complete game_engine.py.

⚙ Edited game_engine.py
⚙ Edited game_engine.py
⚙ Edited game_engine.py
⚙ Edited game_engine.py
⚙ Edited game_engine.py
⚙ Edited game_engine.py
⚙ Ran command: cd "D:/SEM-5/SE/LAB/LAB-4/53_arm_wrestle" && SDL_VIDEODRIVER=dummy python -c " i

Claude

Everything is applied and tested. One problem is left: in a real match the player still wins before running out of stamina. Details are below the file.

⚙ Ran command: cd "D:/SEM-5/SE/LAB/LAB-4/53_arm_wrestle" && cat game/game_engine.py

Claude

I applied all four changes to game_engine.py, and the exhaustion lock works. But there's still a problem: in a real match the player wins before running out of stamina, so the exhaustion warning never appears during normal play.

Changes: - **Exhaustion lock:** a new `player_exhausted` flag turns on when a press drops stamina to 10 or below. It turns off in `update()` once stamina gets back to 30 (`recover_threshold`). `is_exhausted()` now returns this flag, so key presses and the warning still use the same check. - **Refill rate:** stamina now refills by 0.25 per frame instead of 0.8. - **Cleanups:** `reset()` now uses `self.max_stamina` and also clears the new flag, and `import math` is gone. I also reworded one comment about the stamina bar, since it said "below 10" and exhaustion now lasts until 30.

Test results (run without a window): - Once exhausted, input stayed locked for 86 frames (about 1.4 s). Every key press in that time was ignored, and the lock released at exactly 30 stamina. - `reset()` turns the lock off and puts stamina back to 100.

The remaining problem: a full bar allows about 45 presses before exhaustion. That's roughly 190 units of push, but the player only needs 100 to win. I ran 20 matches at each speed from 6 to 15 presses per second. The player won every match (2–11 s long) and never got exhausted.

The fix I'd suggest is raising the stamina cost per press from `2.0` to about `5.0` in both branches of `handle_event()`. At 10 presses per second, the player would tire out after about 2.5 s, before reaching −100. Then they'd be locked out while the computer pushes back, which gives the back-and-forth rhythm the README describes. I haven't changed it, since it changes how the game plays; tell me if you want it. If you only need footage for your video, temporarily setting `self.stamina = 15.0` in `__init__` shows the effect straight away.

Final complete `game_engine.py`:

```
import random
import pygame


class GameEngine:

    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.arm_position = 0.0
        self.target_limit = 100.0
        self.last_key = None

        self.stamina = 100.0
        self.max_stamina = 100.0
        self.exhaust_threshold = 10.0  # at or below this, player input is ignored...
        self.recover_threshold = 30.0  # ...until stamina climbs back to this
        self.player_exhausted = False

        self.winner = None
        self.game_state = "PLAYING"
```

```
        self.ai_strength = 0.35

        # AI surge cycle: BUILDING -> SURGE -> EXHAUSTED -> BUILDING (timers in frames @ 60 FPS)
        self.ai_force_multipliers = {"BUILDING": 1.0, "SURGE": 2.5, "EXHAUSTED": 0.3}
        self.ai_max_energy = 100.0
        self.ai_energy = 0.0
        self.ai_state = "BUILDING"
        self.ai_state_timer = 0

        self.font_big = pygame.font.SysFont(None, 44)
        self.font_med = pygame.font.SysFont(None, 26)

    def handle_event(self, event):
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()
            return

        if event.type == pygame.KEYDOWN:
            if self.is_exhausted():
                return

            # Player wins at -target_limit, so player force must decrease arm_position
            # (the AI in update() adds positive force toward the computer's side).
            if event.key == pygame.K_LEFT:
                if self.last_key != pygame.K_LEFT:
                    self.arm_position -= 4.2
                    self.stamina = max(0.0, self.stamina - 2.0)
                    self.last_key = pygame.K_LEFT
            elif event.key == pygame.K_RIGHT:
                if self.last_key != pygame.K_RIGHT:
                    self.arm_position -= 4.2
                    self.stamina = max(0.0, self.stamina - 2.0)
                    self.last_key = pygame.K_RIGHT

            if self.stamina <= self.exhaust_threshold:
                self.player_exhausted = True

    def is_exhausted(self):
        return self.player_exhausted

    def update(self):
        if self.game_state != "PLAYING":
            return

        self.update_ai_state()

        ai_variance = random.uniform(0.3, 1.0)
        multiplier = self.ai_force_multipliers[self.ai_state]
        self.arm_position += self.ai_strength * ai_variance * multiplier

        if self.stamina < self.max_stamina:
            self.stamina = min(self.max_stamina, self.stamina + 0.25)

        if self.player_exhausted and self.stamina >= self.recover_threshold:
            self.player_exhausted = False
```

```
        if self.arm_position <= -self.target_limit:
            self.winner = "PLAYER"
            self.game_state = "GAME_OVER"
        elif self.arm_position >= self.target_limit:
            self.winner = "COMPUTER"
            self.game_state = "GAME_OVER"

    def update_ai_state(self):
        if self.ai_state == "BUILDING":
            # Charges to full in roughly 2-3 seconds
            self.ai_energy = min(self.ai_max_energy, self.ai_energy + random.uniform(0.55, 0.85))
            if self.ai_energy >= self.ai_max_energy:
                self.ai_state = "SURGE"
                self.ai_state_timer = random.randint(60, 120)  # 1-2 seconds
        elif self.ai_state == "SURGE":
            self.ai_state_timer -= 1
            self.ai_energy = self.ai_max_energy * max(0, self.ai_state_timer) / 120
            if self.ai_state_timer <= 0:
                self.ai_state = "EXHAUSTED"
                self.ai_energy = 0.0
                self.ai_state_timer = random.randint(60, 120)  # 1-2 seconds
        elif self.ai_state == "EXHAUSTED":
            self.ai_state_timer -= 1
            if self.ai_state_timer <= 0:
                self.ai_state = "BUILDING"
                self.ai_state_timer = 0

    def reset(self):
        self.arm_position = 0.0
        self.stamina = self.max_stamina
        self.player_exhausted = False
        self.last_key = None
        self.winner = None
        self.game_state = "PLAYING"
        self.ai_energy = 0.0
        self.ai_state = "BUILDING"
        self.ai_state_timer = 0

    def render(self, screen):
        screen.fill((25, 28, 35))

        title_surf = self.font_big.render("ARM WRESTLE SHOWDOWN", True, (240, 240, 240))
        screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 12))

        player_header = self.font_med.render("PLAYER", True, (80, 160, 255))
        computer_header = self.font_med.render("COMPUTER", True, (255, 100, 80))
        screen.blit(player_header, (60, 55))
        screen.blit(computer_header, (self.width - 150, 55))

        ai_state_colors = {
            "BUILDING": (230, 200, 90),
            "SURGE": (255, 60, 60),
            "EXHAUSTED": (140, 150, 170),
        }
        ai_label = "SURGE!" if self.ai_state == "SURGE" else self.ai_state
```

```
        ai_state_surf = self.font_med.render(ai_label, True, ai_state_colors[self.ai_state])
        screen.blit(ai_state_surf, (self.width - 150, 78))

        table_rect = pygame.Rect(40, 100, self.width - 80, 310)
        pygame.draw.rect(screen, (110, 50, 15), table_rect, border_radius=14)
        pygame.draw.rect(screen, (70, 30, 8), table_rect, width=5, border_radius=14)

        pygame.draw.line(screen, (45, 18, 4), (self.width // 2, 100), (self.width // 2, 410), 4)

        offset_x = (self.arm_position / self.target_limit) * 95
        hand_x = (self.width // 2) + int(offset_x)
        hand_y = 235

        p_shoulder = (70, 330)
        p_elbow = (140, 215)
        c_shoulder = (self.width - 70, 330)
        c_elbow = (self.width - 140, 215)

        exhausted = self.is_exhausted() and self.game_state == "PLAYING"
        ticks = pygame.time.get_ticks()

        # Tremble the player's arm while exhausted. The hand stays put because the
        # computer's arm is attached to it too.
        if exhausted:
            p_shoulder = (p_shoulder[0] + random.randint(-1, 1), p_shoulder[1] + random.randint(-1, 1))
            p_elbow = (p_elbow[0] + random.randint(-3, 3), p_elbow[1] + random.randint(-3, 3))

        pygame.draw.line(screen, (200, 145, 110), p_shoulder, p_elbow, 32)
        pygame.draw.line(screen, (215, 160, 125), p_elbow, (hand_x, hand_y), 26)
        pygame.draw.circle(screen, (185, 130, 95), p_elbow, 18)

        pygame.draw.line(screen, (170, 110, 85), c_shoulder, c_elbow, 32)
        pygame.draw.line(screen, (185, 125, 95), c_elbow, (hand_x, hand_y), 26)
        pygame.draw.circle(screen, (150, 95, 70), c_elbow, 18)

        pygame.draw.circle(screen, (225, 175, 140), (hand_x, hand_y), 24)
        pygame.draw.circle(screen, (160, 115, 85), (hand_x, hand_y), 24, width=3)

        stamina_label = self.font_med.render("STAMINA", True, (220, 220, 220))
        screen.blit(stamina_label, (40, 445))

        stamina_bg = pygame.Rect(140, 448, 240, 22)
        stamina_fill = pygame.Rect(140, 448, int(240 * (self.stamina / self.max_stamina)), 22)
        if exhausted:
            # Flash the whole bar, since the fill is small while exhausted
            flash_on = (ticks // 150) % 2 == 0
            bg_color = (150, 25, 25) if flash_on else (45, 50, 60)
            bar_color = (255, 70, 70) if flash_on else (120, 20, 20)
        else:
            bg_color = (45, 50, 60)
            bar_color = (60, 210, 100) if self.stamina > 25 else (220, 60, 60)
        pygame.draw.rect(screen, bg_color, stamina_bg, border_radius=6)
        pygame.draw.rect(screen, bar_color, stamina_fill, border_radius=6)

        if exhausted and (ticks // 300) % 2 == 0:
            exhausted_surf = self.font_med.render("EXHAUSTED!", True, (255, 70, 70))
```

```
            screen.blit(exhausted_surf, (stamina_bg.right + 15, 450))

        if self.game_state == "GAME_OVER":
            overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
            overlay.fill((0, 0, 0, 200))
            screen.blit(overlay, (0, 0))

            win_text = "PLAYER WINS THE MATCH!" if self.winner == "PLAYER" else "COMPUTER WINS!"
            color = (80, 240, 100) if self.winner == "PLAYER" else (240, 80, 80)
            text_surf = self.font_big.render(win_text, True, color)
            screen.blit(
                text_surf,
                (self.width // 2 - text_surf.get_width() // 2, self.height // 2 - 45)
            )

            restart_surf = self.font_med.render(
                "Press [R] to Rematch", True, (240, 240, 240)
            )
            screen.blit(
                restart_surf,
                (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 10)
            )
```